I need some help with my work with R using Positron

Positron is Posit's next-generation integrated development environment (IDE) built specifically for data science in both R and Python. It combines the modern editor experience of VS Code with the familiar data science workflows traditionally found in RStudio.[1] [2]

Setup and Configuration

Positron automatically discovers and manages your existing R installations on your machine [3]. You can easily customize your R coding environment using user or workspace settings. For example, Positron sets the default R tab size to 2 spaces, but you can override this and other defaults by editing your `settings.json` file [3]. You can also specify which pipe operator you prefer to insert (`|>` or `%>%`) when using keyboard shortcuts [3].

Key Features for R Workflows

- **Data Explorer:** This tool provides an interactive, spreadsheet-like grid for inspecting `data.frames` and `tibbles`. It features a summary panel for checking variable types, built-in sorting and filtering, and the ability to copy selected rows directly into spreadsheet applications.[4]
- **Dedicated Plotting:** Positron includes a distinct editor window for inspecting visualizations. You can seamlessly resize your plots to specific aspect ratios, scroll through your entire plot history, and copy graphics directly to your clipboard.[4]
- **Advanced Debugging:** The IDE supports a modern debugging workflow, allowing you to set breakpoints, step through your R code, and inspect active variables in the environment pane.[3]
- **Keyboard Shortcuts:** Positron inherits VS Code's shortcuts but adds R-specific bindings, such as inserting the assignment operator (`<-`) or instantly loading the current R package using `devtools::load_all()`.[3]

Learning Resources

To get hands-on practice, you can install the `positron.tutorials` package in R. By downloading the development version from GitHub (`PPBDS/positron.tutorials`), you gain access to interactive lessons. Using the command `learnr::run_tutorial()`, you can launch tutorials covering basic R scripts, Quarto document creation, and Git integration specifically tailored for the Positron interface.[5] [6]

What specific aspect of using R in Positron are you trying to figure out right now—such as configuring settings, exploring data, or generating plots?

I noticed that in sf, st_is_valid() returns a lot of TRUE and a couple of FALSE. I tried to use the count command to count the number of invalid geometries, like so,

count(!st_is_valid(country_boundaries))

However, I ran into an error, "Error in `UseMethod()`:
! no applicable method for 'count' applied to an object of class "logical""

What is the appropriate method to count the number of invalid geometries?

The error occurs because `dplyr::count()` expects a data frame or tibble, but `st_is_valid()` returns a simple logical vector of `TRUE` and `FALSE` values. [16]

To count the number of invalid geometries, you can take advantage of how R handles logical vectors: it treats `TRUE` as 1 and `FALSE` as 0.

Here are the most appropriate methods depending on what you want to achieve:

Simply Count the Total Invalid Geometries

Since you want to find the number of invalid geometries (where `st_is_valid()` returns `FALSE`), you can use the `sum()` function on the negated vector:

```
sum(!st_is_valid(country_boundaries))
```

This adds up all the `TRUE` values (which represent the `FALSE` invalid results from `st_is_valid()`), giving you the exact total of invalid geometries.

See the Breakdown of Valid vs Invalid

If you want to see a tally of both valid and invalid geometries side-by-side using base R, use the `table()` function:

```
table(st_is_valid(country_boundaries))
```

Use `count()` within a Data Frame Context

If you specifically want to use `dplyr::count()`, you need to apply it directly to the dataset itself, rather than the vector output. You can achieve this by using the `st_is_valid()` check inside the `count()` function:

```
library(dplyr)

country_boundaries %>%
  count(is_valid = st_is_valid(geometry))
```

This will return a tibble showing the counts for both `TRUE` (valid) and `FALSE` (invalid) geometries.

Do you also need help repairing these invalid geometries so you can perform spatial operations on them?

⁂

How do I use st_cast()?

This was how I used to it to try to cast a MULTIPOLYGON to only "POLYGON"
"no_of_polygons <- st_cast(singapore, to, 'POLYGON')"

But I ran into this error,
Error:
! object 'to' not found
Backtrace:
■
1. ├─sf::st_cast(singapore, to, "POLYGON")
2. └─sf:::st_cast.sf(singapore, to, "POLYGON")
3. ├─sf::st_cast(st_geometry(x), to, group_or_split = do_split)
4. └─sf:::st_cast.sfc(st_geometry(x), to, group_or_split = do_split)
5. └─sf:::which_sfc_col(to)

Quitting from week-10-lab.qmd:139-150 [count-points-singapore]
Execution halted
WARN: Error encountered when rendering files

The error you're encountering, `object 'to' not found`, happens because you wrote the target geometry type without using quotation marks or as an actual parameter argument. When calling `st_cast()`, R is looking for an object named `to` instead of understanding it as the parameter name. [26]

Here is the correct syntax for your `st_cast()` function:

```
no_of_polygons <- st_cast(singapore, "POLYGON")
```

Alternatively, if you want to explicitly name the parameter, you can write:

```
no_of_polygons <- st_cast(singapore, to = "POLYGON")
```

Important Note About Casting

When you cast a `sf` object containing `MULTIPOLYGON` geometries to `POLYGON`, `st_cast()` will "split" the multipolygons. This means if one `MULTIPOLYGON` feature previously contained 3 separate polygons, casting it will result in 3 separate `POLYGON` features (rows) in your `sf` dataframe, with the attribute data from the original row duplicated for each new polygon part. [27] [28]

Are you looking to break apart all your separate islands into individual features, or were you hoping to preserve the current row count in your spatial dataframe?

⁂

how does the "keep = 0.05," and "weighting = 0.7," work for ms_simplify()?

In the `rmapshaper::ms_simplify()` function, the `keep` and `weighting` parameters control how aggressively your spatial polygons are simplified, specifically when using the default Visvalingam algorithm. [36]

The `keep` Parameter

The `keep` parameter accepts a proportion between 0 and 1 representing the percentage of vertices (points) from the original geometry that you want to retain. [36]

By setting `keep = 0.05` (the default value), you are telling the function to keep only 5% of the original vertices and discard 95% of them. This drastically reduces the file size and complexity of the geometry, which speeds up plotting. However, depending on how detailed the original shape was, reducing the points to 5% could cause smaller features (like small islands) to disappear entirely. [37] [38] [36]

The `weighting` Parameter

The `weighting` parameter only applies when using the Visvalingam simplification algorithm (the default). It accepts a numeric value, with a default of 0.7. [36]

The Visvalingam algorithm works by calculating the effective area of triangles formed by adjacent vertices and removing the ones with the smallest areas. The `weighting` parameter adjusts this area calculation by underweighting the area of triangles that form acute angles. [36]

- Setting `weighting = 0.7` smooths out the sharpest, most jagged edges of your polygons. [36]
- A higher value (e.g., `0.9`) will produce a much smoother, rounded appearance. [38]
- Setting `weighting = 0` completely disables this feature, meaning the algorithm will evaluate triangles purely based on their mathematical area, without considering whether the angle is sharp or flat. [36]

If you find that `keep = 0.05` is deleting entire polygons from your dataset (like tiny islands in Singapore), you may want to know how to prevent those polygons from disappearing while still

simplifying the larger ones. Would you like to know how to force `ms_simplify()` to keep all distinct shapes regardless of the `keep` threshold?

⁂

If I am looking to retain every country, which arguments should I focus on?

If you are aggressively simplifying a map (using a low `keep` value) but want to guarantee that no entire country disappears, you need to use the `keep_shapes` argument. [51] [52]

By default, `keep_shapes = FALSE`. This means that if a polygon becomes too small during simplification, the algorithm will simply delete it.

To prevent this, you should set it to `TRUE`:

```
ms_simplify(country_boundaries, keep = 0.05, keep_shapes = TRUE)
```

This forces `ms_simplify()` to retain at least one polygon ring per feature, ensuring that every row in your dataset remains visible on the map. [53]

Handling Multi-Part Countries (Islands)

There is a catch when dealing with countries made up of multiple islands (Multipolygons). If you only set `keep_shapes = TRUE`, the function will guarantee that the *country* survives, but it will only retain the largest island (the largest part of the multi-part polygon) while smaller islands may still be deleted. For example, New Zealand might be reduced to just the South Island. [54] [55]

If you want to ensure that *every single island* belonging to a country is retained, you must use the `explode` argument as well:

```
ms_simplify(country_boundaries, keep = 0.05, keep_shapes = TRUE, explode = TRUE)
```

Setting `explode = TRUE` temporarily breaks all multipart polygons into single, individual polygons before simplifying. Combined with `keep_shapes = TRUE`, this guarantees that every single tiny island will survive the simplification process, no matter how low your `keep` value is. [55] [54]

Do you need help checking how the final map looks, or calculating the size difference before and after simplification?

⁂

What do the arguments for st_wrap_dateline mean?

This was provided in the help argument under st_transform

st_wrap_dateline(x, options, quiet)

The `st_wrap_dateline()` function in the `sf` package is used to correctly handle geometries (like polygons or lines) that cross the international dateline (180 degrees longitude). When these features are plotted natively, they often draw a straight line all the way across the globe to connect points, causing visual artifacts.

Here is what the three arguments in `st_wrap_dateline(x, options, quiet)` mean:

`x`

This is your input data. It must be an object of class `sf`, `sfc`, or `sfg` (the standard spatial objects in the `sf` package).[66]

`options`

This argument accepts a character vector of configuration options passed directly to the underlying GDAL library, which actually performs the math to slice your geometry.[67] [68]
The most common options you will pass here are:

- `"WRAPDATELINE=YES"`: This is the core instruction telling GDAL to actively cut the geometries that span the dateline.[69] [67]
- `"DATELINEOFFSET=..."`: This specifies the tolerance (in degrees) for how far a geometry needs to stretch before it is considered to be "crossing the dateline." For example, setting `"DATELINEOFFSET=180"` tells the algorithm that a line must span at least 180 degrees of longitude for it to be cut.[70] [67]

Usually, you will pass these together as a combined vector:
`options = c("WRAPDATELINE=YES", "DATELINEOFFSET=180")`.[67] [70]

`quiet`

This is a logical argument (`TRUE` or `FALSE`).

- If set to `FALSE` (the default), the function may print GDAL-related progress updates, warnings, or diagnostic messages to your console.[70]
- If set to `TRUE`, it suppresses these messages so your console stays clean. This is especially useful if you are rendering an R Markdown or Quarto document and don't want function chatter appearing in your final output.[68]

Are you dealing with an issue where a polygon is rendering as a strange straight horizontal line across your entire map?

⁂

Does st_wrap_dateline result in a polygon?

Yes, when applied to a polygon that crosses the dateline, `st_wrap_dateline()` typically results in a polygon, but its specific geometry class changes.

When the function successfully "wraps" (or cuts) a single `POLYGON` that crosses the dateline, it splits the shape into two or more separate polygon pieces—one on the far east side of the map (near 180° longitude) and one on the far west side (near -180° longitude).

Because a single row in your dataset now represents multiple distinct shapes, `st_wrap_dateline()` automatically converts the original `POLYGON` into a `MULTIPOLYGON`. [76] [77]

Do you need help checking if your geometries are currently `POLYGON` or `MULTIPOLYGON` after running the function?

⁂

Currently I have found out the codes that have duplicated features which are derived from this,

```
duplicate_countries <- count_code |>
  select(code) |>
  filter(code != "NA") |>
  st_drop_geometry()
```

And I am currently looking to merge the same-code features into one feature, and this was what I was trying to do

```
country_boundaries <- country_boundaries |>
  summarize(
    status = ifelse(any(status == "Member State"),
           "Contains a Member State",
           "Does Not Contain a Member State"),
    .by = duplicate_countries
  ) |>
  st_cast(country_boundaries, to = 'MULTIPOLYGON').
```

How do I target only these duplicate countries, while leaving the non-duplicate ones intact?

To target only the countries with duplicate codes for merging while leaving the other (non-duplicate) country boundaries exactly as they are, you should split your workflow into three steps:

1. Filter the main dataset into two separate datasets: one containing only the duplicates, and one containing the non-duplicates.
2. Perform the `summarize()` and `st_cast()` operations only on the duplicated dataset to merge them.

3. Bind the newly merged duplicates back together with the non-duplicate dataset.

Here is how you can write this out using your existing objects:

```
library(dplyr)
library(sf)

# 1. Split the data into duplicates and non-duplicates
# Keep features where the code is in your duplicate_countries list
duplicate_data <- country_boundaries |>
    filter(code %in% duplicate_countries$code)

# Keep features where the code is NOT in your duplicate_countries list
non_duplicate_data <- country_boundaries |>
    filter(!code %in% duplicate_countries$code)

# 2. Merge geometries for the duplicates ONLY
merged_duplicates <- duplicate_data |>
    summarize(
        status = ifelse(any(status == "Member State"),
                        "Contains a Member State",
                        "Does Not Contain a Member State"),
        .by = code # group by the code instead of the dataset itself
    ) |>
    st_cast("MULTIPOLYGON")

# 3. Combine them back together
# Note: Ensure that the columns in 'merged_duplicates' match those in 'non_duplicate
# If summarize dropped other important columns, you may need to add them back before
final_boundaries <- bind_rows(non_duplicate_data, merged_duplicates)
```

Important note on your `summarize` step:
In your original code, you wrote `.by = duplicate_countries`. The `.by` argument expects the unquoted name of a column inside your dataframe (like `code`), not an external dataframe object. Changing this to `.by = code` ensures `sf` correctly groups and merges the geometries that share the same code.[86] [87]

Do you have other columns in your `country_boundaries` dataset (like country names or populations) that you need to preserve when you run the `summarize()` function?

⁂

What is bind_rows and is it safe to interact with sf objects?

`bind_rows()` is a function from the `dplyr` package that is used to stack multiple data frames on top of each other, binding them by their rows. It is incredibly useful because, unlike base R's `rbind()`, `bind_rows()` allows you to merge datasets that do not have perfectly matching columns. If one dataset has a column that the other does not, `bind_rows()` will still merge them and simply fill the missing column in the other rows with `NA`.[96]

Using `bind_rows()` with `sf` objects

Yes, `bind_rows()` is currently completely safe to use with `sf` objects. [97] [98]

However, this was not always the case. In earlier versions of `dplyr` (prior to version 1.0.0 released in 2020), `bind_rows()` stripped the spatial metadata (the geometry column and CRS) from `sf` objects, causing errors when users tried to combine spatial datasets. Back then, the community had to rely on `rbind()` or specialized workarounds. [98]

Since `dplyr` v1.0.0, the package authors fully integrated support for `sf` objects natively. When you use `bind_rows()` on two `sf` objects today: [97]

- It correctly merges the attribute tables.
- It preserves the geometry column (usually `geometry`).
- It ensures the coordinate reference system (CRS) remains intact.

A Crucial Warning

To use `bind_rows()` safely on `sf` objects, **both objects must share the exact same Coordinate Reference System (CRS)**. If you attempt to bind a dataset mapped in WGS84 (EPSG:4326) to one projected in Web Mercator (EPSG:3857), you will encounter errors or silent geometric misalignments. You should always ensure they match (using `st_crs()`) before binding them together.

Are you running into any specific column mismatch errors when you try to bind your duplicated and non-duplicated datasets back together?

⁂

ggplot(country_boundaries) +

geom_sf() +
  coord_sf(srid = "EPSG:8857")

Why am I getting the error, Error in `coord_sf()`:
! unused argument (srid = "EPSG:8857")

You are getting this error because `srid` is not a valid argument for `coord_sf()`.

In earlier versions of the `sf` package and `ggplot2`, there was sometimes confusion about how to specify coordinate reference systems (CRS). However, `coord_sf()` uses the argument `crs` to define the target projection. [106]

To fix this error, simply change `srid` to `crs` like this:

```
ggplot(country_boundaries) +
    geom_sf() +
    coord_sf(crs = "EPSG:8857")
```

When you provide the string `"EPSG:8857"`, `coord_sf()` will automatically parse it and transform your map into the Equal Earth projection on the fly. [107] [108]

Do you need help adjusting the `xlim` and `ylim` arguments in `coord_sf()` to zoom into a specific area on your map?

⁂

Currently I tried plotting a Mollweide projection, however I kind of noticed that it is "incomplete" at the top where the lines are not closed off

The incomplete, "open" graticules at the top and bottom of your Mollweide projection are a known rendering artifact when using `coord_sf()` and `geom_sf()` with certain global map projections in R. [116] [117]

When `ggplot2` draws the background grid (graticules), it relies on the bounding box of the data provided. In global projections like Mollweide or Robinson, the calculation sometimes fails to fully trace the "edge of the globe" where the bounding box pinches at the poles. [117] [116]

Here are the best ways to fix this so your map looks complete:

Method 1: The `sfhelper` Package

The easiest way to force the bounding outline to draw completely is to use the `st_transform_outline()` function from the `sfhelper` package. This creates an explicit `sf` polygon that perfectly traces the outer edge of your chosen projection, which you can use as a background layer (e.g., to represent the ocean). [117]

```
library(sfhelper)
library(ggplot2)

moll_crs <- "+proj=moll" # define your CRS
world_outline <- st_transform_outline(crs = moll_crs)

ggplot() +
    # Draw the complete global bounding box first
    geom_sf(data = world_outline, fill = "aliceblue", color = "black") +
    # Draw your map on top
    geom_sf(data = country_boundaries) +
    coord_sf(crs = moll_crs)
```

Method 2: Expand the Coordinate Limits

Sometimes the graticules get cut off simply because `coord_sf()` trims the map too tightly against the bounding box of your country polygons. You can try forcing the map slightly wider by using `coord_sf(expand = FALSE)` combined with manually defined limits. However, this method involves trial and error to find the right `xlim` and `ylim` values for your specific CRS. [118]

Are you also experiencing issues with broken or distorted country polygons wrapping across the map after changing to this projection, or are only the background lines affected?

⁂

What if I were to set limits in coord_sf() instead? Would it be able to help with the issue?

Yes, setting the limits in `coord_sf()` can help, but it requires a bit of manual tuning and comes with some quirks depending on your map projection.

How to Set Limits

You can define the `xlim` and `ylim` arguments inside `coord_sf()`. By default, `ggplot2` adds a small buffer around the map (controlled by the `expand` argument). To fix the broken edge of your Mollweide projection, you generally want to turn off this expansion buffer and set exact limits. [127] [128]

For example, to bound a global Mollweide map, you would use coordinates matching the extent of your projection:

```
ggplot(country_boundaries) +
  geom_sf() +
  coord_sf(
    crs = "+proj=moll",
    xlim = c(-18000000, 18000000), # Projected coordinates, not degrees
    ylim = c(-9000000, 9000000),
    expand = FALSE
  )
```

Caveats to the Limit Method

While setting limits can force `ggplot2` to draw the graticules across the entire specified box, there are a few things to watch out for:

1. **Coordinate Units:** When you change the CRS in `coord_sf()`, the limits must be passed in the units of that target projection. For Mollweide, this means you must provide large meter-based coordinates, not standard longitude/latitude degrees. [128]
2. **Missing Labels:** Sometimes, if you manually set the bounding box too tight or too loose, `coord_sf()` may silently drop the graticule axis labels (the degree markings on the side of the map). [129]
3. **Blank Maps:** If you input standard degree coordinates (e.g., `-180` to `180`) into a map projected in meters, the limits will zoom into a microscopically small, empty point in the ocean, resulting in a blank map. [130]

If you try this and the map turns completely blank or you lose your axis labels, using the `sfhelper` method mentioned previously is often the more reliable workaround for global projections.

Are you looking to show the entire world map, or are you trying to zoom in on a specific continent using the Mollweide projection?

⁂

Currently I am looking to choose a sequential palette, what is the command to check the sequential palettes offered

If you want to explore the available sequential color palettes, you have two primary options in R depending on which package you are using.

Using RColorBrewer (Most Common)

If you are using the popular `RColorBrewer` package (which is built into `ggplot2`'s `scale_color_brewer()` and `scale_fill_brewer()` functions), you can visually check all available palettes, including the sequential ones, by running:

```
library(RColorBrewer)
display.brewer.all(type = "seq")
```

This command will draw a plot in your viewer pane showing the names and colors of all the sequential palettes (such as "Blues", "Greens", "Oranges", etc.).[142]

Using the `colorspace` Package

If you are using the modern `colorspace` package (which provides excellent HCL-based color palettes via functions like `scale_color_continuous_sequential()`), you can list the names of all the available sequential palettes by running:

```
library(colorspace)
hcl_palettes(type = "sequential")
```

To visually inspect these `colorspace` palettes instead of just seeing their names, you can use:

```
hcl_palettes(type = "sequential", plot = TRUE)
```

This will display a beautiful chart of all the sequential options.[143]

Are you applying these colors to a continuous variable (like temperature) or a discrete/categorical variable (like low, medium, and high)?

⁂

I tried adding a new geom_sf() layer like so,

choropleth_map <- choropleth_map + geom_sf(data = filter(countries, is.na(gdp_per_cap)), aes(color = "Missing"), fill = "lightgray")

But it gave me this warning,
Coordinate system already present.
i Adding new coordinate system, which will replace the existing one.

This then remove the previous layers of choropleth map that I had,

choropleth_map <- ggplot(countries) + geom_sf(aes(fill = gdp_per_cap)) + coord_sf(crs = "EPSG:8857")

choropleth_map <- choropleth_map + scale_fill_fermenter(palette = "Reds", breaks = c(2000, 5000, 10000, 20000, 50000, 100000), labels = label_number(scale_cut = cut_short_scale()))

The warning occurs because `geom_sf()` implicitly calls `coord_sf()` under the hood if it isn't already present, or if it thinks a new spatial layer requires it.[152]

In `ggplot2`, a plot can only have **one** coordinate system. When you added a new `geom_sf()` layer *after* having already defined `coord_sf(crs = "EPSG:8857")` in your original `choropleth_map` object, `ggplot2` triggered a new `coord_sf()` default, overwriting the one you had explicitly set. This causes your map to revert to the default CRS of the data and can drop previously defined limits or properties.[153] [154] [155]

How to Fix It

To avoid this issue, you should define your custom `coord_sf(crs = "EPSG:8857")` as the **very last** layer in your ggplot chain, after all `geom_sf()` layers have been added.

Here is how you should structure your code:

```
choropleth_map <- ggplot(countries) +
    # 1. Base layer
    geom_sf(aes(fill = gdp_per_cap)) +

    # 2. Add the missing data layer
    geom_sf(
        data = filter(countries, is.na(gdp_per_cap)),
        aes(color = "Missing"),
        fill = "lightgray"
    ) +

    # 3. Add your scales
    scale_fill_fermenter(
        palette = "Reds",
        breaks = c(2000, 5000, 10000, 20000, 50000, 100000),
        labels = label_number(scale_cut = cut_short_scale())
    ) +
```

```
    # 4. Define the coordinate system LAST
    coord_sf(crs = "EPSG:8857")
```

By placing `coord_sf()` at the end, you ensure that it is the final authority on the coordinate system for the entire plot, preventing `ggplot2` from quietly overwriting it. [152]

Are you getting the gray fill for your missing data correctly, or does the scale legend look messy now that you added the `"Missing"` color aesthetic?

⁂

Currently the task is to Fill the countries whose GDP value is missing with a light gray and the undetermined areas with a darker gray, and give each gray its own legend key.

Is this the correct way to be doing this or do I also have to edit the base choropleth layer?

Yes, using separate layers is actually the correct strategy!

In `ggplot2`, you are strictly allowed to have only **one scale per aesthetic**. This means you cannot map continuous/binned numbers (your GDP) and discrete categories ("Missing", "Undetermined") to the exact same `fill` scale.

However, your current code snippet has a small flaw: by writing `aes(color = "Missing")` and hardcoding `fill = "lightgray"` outside of the `aes()`, `ggplot2` will create a legend key for the polygon *borders* (color), but the *fill* will not get a legend at all.

Here are the two best ways to solve this and give each gray its own fill legend key:

Method 1: The `ggnewscale` Package (Recommended)

The cleanest way to do this is to use the `ggnewscale` package. It literally allows you to reset the `fill` scale halfway through your code so you can add a second one for your missing data.

```
# Install if you haven't: install.packages("ggnewscale")
library(ggnewscale)

choropleth_map <- ggplot(countries) +
    # 1. Base layer (Continuous/Binned GDP)
    geom_sf(aes(fill = gdp_per_cap)) +
    scale_fill_fermenter(
        palette = "Reds",
        breaks = c(2000, 5000, 10000, 20000, 50000, 100000),
        labels = label_number(scale_cut = cut_short_scale()),
        na.value = "transparent" # Hide NAs so the next layers can show them
    ) +

    # 2. Reset the fill scale!
```

```
    new_scale_fill() +

    # 3. Add the Missing layer
    geom_sf(
        data = filter(countries, is.na(gdp_per_cap) & status != "Undetermined"),
        aes(fill = "Missing") # Now mapped to fill inside aes()
    ) +

    # 4. Add the Undetermined layer
    geom_sf(
        data = filter(countries, is.na(gdp_per_cap) & status == "Undetermined"),
        aes(fill = "Undetermined")
    ) +

    # 5. Define the new discrete fill scale for the missing/undetermined data
    scale_fill_manual(
        name = "No Data",
        values = c("Missing" = "lightgray", "Undetermined" = "darkgray")
    ) +

    # 6. Coordinate system MUST be last
    coord_sf(crs = "EPSG:8857")
```

(Note: Adjust the `status == "Undetermined"` *filter above to match whatever logic you are using to identify those areas).*

Method 2: The Base `ggplot2` Hack

If you cannot install new packages, your original idea of mapping to `color` is actually a known workaround. You map the discrete layers to the *border color*, make the actual borders invisible, and then force the legend to draw filled squares using `override.aes`.

```
choropleth_map <- ggplot(countries) +
    geom_sf(aes(fill = gdp_per_cap)) +
    scale_fill_fermenter(
        palette = "Reds",
        breaks = c(2000, 5000, 10000, 20000, 50000, 100000),
        labels = label_number(scale_cut = cut_short_scale())
    ) +

    # Missing layer: hardcode fill, map color
    geom_sf(
        data = filter(countries, is.na(gdp_per_cap) & status != "Undetermined"),
        aes(color = "Missing"), fill = "lightgray"
    ) +

    # Undetermined layer
    geom_sf(
        data = filter(countries, is.na(gdp_per_cap) & status == "Undetermined"),
        aes(color = "Undetermined"), fill = "darkgray"
    ) +

    # Map the "color" to transparent so map borders aren't affected,
    # but override the legend keys to look like filled boxes
```

```
    scale_color_manual(
        name = "No Data",
        values = c("Missing" = "transparent", "Undetermined" = "transparent"),
        guide = guide_legend(override.aes = list(
            fill = c("lightgray", "darkgray"),
            color = "black" # Optional: gives the legend keys a black border
        ))
    ) +
    coord_sf(crs = "EPSG:8857")
```

Method 1 is much easier to read and maintain, but Method 2 is great to know if you want to stick strictly to `ggplot2`.

Are your missing and undetermined data points stored in the same dataset, or are you pulling the undetermined geometries from somewhere else?

Currently I am looking to "Clip the maps at 56°S and 84°N." Would using something like

scale_x_continuous(breaks = seq(-180, 180, by = 45)) +
  scale_y_continuous(
    breaks = c(-89.9, -45, 0, 45, 89.9)
)

work?

No, using `scale_y_continuous()` limits will not correctly clip a spatial map using `geom_sf()`.[162] [163]

In `ggplot2`, the `scale_x_continuous()` and `scale_y_continuous()` functions are used to set axis breaks (the labels shown on the side). While they do have a `limits` argument, applying limits through standard scales on `sf` spatial objects will often delete polygons that partially overlap the boundary rather than cleanly slicing them, or it will throw an error entirely.[164] [162]

How to correctly clip the map

To safely clip a spatial map in `ggplot2` without losing geometries, you must apply the limits inside the coordinate system function: `coord_sf()`.

Since your base map is projected using `coord_sf(crs = "EPSG:8857")` (Equal Earth, which uses meters, not degrees), you cannot just pass raw degrees like `-56` and `84` into the `ylim` argument. You must first convert those latitude bounds into the corresponding Y-coordinates for EPSG:8857.[163]

If your map was strictly using normal longitude/latitude (EPSG:4326), you could clip it like this:

```
choropleth_map +
    coord_sf(ylim = c(-56, 84))
```

However, since you are using EPSG:8857, the easiest way to clip it is actually to crop the spatial data *before* giving it to ggplot, using `sf::st_crop()`.

Cropping with `st_crop`

```
# 1. Crop your spatial data by providing a bounding box in degrees
clipped_countries <- st_crop(
    countries,
    xmin = -180, xmax = 180,
    ymin = -56, ymax = 84
)

# 2. Then plot the clipped data
ggplot(clipped_countries) +
    geom_sf(aes(fill = gdp_per_cap)) +
    coord_sf(crs = "EPSG:8857")
```

Using `st_crop()` physically cuts the polygons at your specified lines of latitude. When you pass this newly clipped dataset into `ggplot2`, the map will automatically adjust its view to fit the remaining data. [163]

Would you like help calculating the exact meter-based limits for `coord_sf()` instead so you don't have to alter your base dataset?

⁂

What other configurations can I make within coord_sf()

Besides changing the projection with `crs` and setting boundaries with `xlim` and `ylim`, `coord_sf()` provides several other useful configuration parameters to customize how your map looks and behaves.

Here are the most common options you can configure within `coord_sf()`:

1. Controlling the Grid (Graticules)

By default, `coord_sf()` draws a background grid of latitude and longitude lines called graticules. You can control how these lines are calculated or turn them off completely.

- `datum`: This dictates which coordinate reference system is used to draw the graticules and axis labels. It defaults to WGS84 (EPSG:4326), meaning you usually see normal latitude/longitude labels on the axes even if your map is projected in meters. If you set `datum = NA`, the map will remove the graticule lines and axis coordinates completely, leaving a clean background. [172]
- `ndiscr`: This controls the "smoothness" of the curved graticule lines. By default, it breaks the lines into 100 small segments (`ndiscr = 100`). If your grid lines look jagged, you can increase this number.

2. Axis Formatting

You can explicitly tell `ggplot2` which sides of the plot should display axis labels (the degrees).

- `label_axes`: Instead of displaying labels on the left and bottom by default, you can provide a list to specify where you want them. For example, `label_axes = list(bottom = "E", left = "N", top = "E", right = "N")` will put labels on all four sides.[172]
- `label_graticule`: This allows you to specifically format the axis tick marks and degree symbols.

3. Padding and Plot Expansion

- `expand`: Just like in standard Cartesian coordinate limits, `ggplot2` adds a small buffer of empty space around your map by default (`expand = TRUE`). If you want your map to perfectly touch the edges of the plot panel, you can set `expand = FALSE`.[172]

4. Default Coordinate System

- `default_crs`: If you are mixing standard `ggplot2` geoms (like `geom_point()`, `geom_line()`, or `annotate()`) on top of your `sf` map, `coord_sf()` will assume those coordinates are already in the same CRS as your map. If your points are stored in standard longitude/latitude but your map is projected in EPSG:8857, you can set `default_crs = 4326` so `coord_sf()` automatically translates your raw X/Y points to match the map projection.[173]

Do you need help applying `datum = NA` to clean up your background for a final publication map?

⁂

Currently I am going to use patchwork to have two panels, one with gdp, which we just plotted and another with life expectancy, if we apply the ylim to only the life expectancy plot will it affect the gdp plot that we had? I am looking to not make any changes to the code I had and only introduce clipping now

No, applying `ylim` inside `coord_sf()` on the life expectancy plot will **not** affect the GDP plot when you combine them using `patchwork`.

In `patchwork`, each plot retains its own completely independent `coord_*` systems, scales, and data. You are essentially just taking two finalized, self-contained images and sticking them side-by-side.

If you apply `coord_sf(ylim = c(ymin, ymax))` only to the code block generating the life expectancy map, the resulting `patchwork` layout will simply show the full global GDP map on one side and the clipped life expectancy map on the other.

A Note on Layout Alignment

While `patchwork` will combine them cleanly, be aware that placing a clipped map next to an unclipped map might result in visual size discrepancies. Because one map is taller (covering the whole globe) and the other is shorter (clipped at the poles), `patchwork` will naturally try to align their bounding boxes. This often causes the clipped map to stretch vertically or scale differently than the full map to fill the available panel space.

If you want them to align perfectly and look identical in size, you would need to apply the exact same clipping limits to both plots.

Do you need help adjusting the `patchwork` layout options (like widths or heights) to handle the different sizes of the two maps?

What about labels? Would there be two different labels if each plot was to have their own labels? Or should I be using patchwork to put the labels

Yes, if each plot has its own title, subtitle, or caption defined inside their respective `ggplot()` code (using `labs()` or `ggtitle()`), those individual labels will be preserved when you combine them with `patchwork`.

However, if you want to add **global labels** (like a single main title for the entire two-panel image, or scientific "A" and "B" tags for each panel), you should use `patchwork`'s `plot_annotation()` function.[187]

Here is how you handle both scenarios:

Individual Plot Labels

If you want the GDP map and the Life Expectancy map to simply have their own titles above them, you define that in the `ggplot` code before combining them:

```
gdp_map <- choropleth_map + labs(title = "Global GDP")
life_exp_map <- life_exp_plot + labs(title = "Life Expectancy")

# Combine them side-by-side
gdp_map + life_exp_map
```

Global Labels using Patchwork

If you want to add a main title over *both* plots, or you want to automatically label them "A" and "B" for a report, you add `plot_annotation()` to the final `patchwork` combination:[187]

```
library(patchwork)

# Combine and add global annotations
gdp_map + life_exp_map +
```

```
    plot_annotation(
        title = "Global Economic and Health Indicators",
        caption = "Data Source: World Bank 2026",
        tag_levels = 'A' # Automatically adds "A" to the first plot and "B" to the se
    )
```

Are you looking to create a shared main title, or do you just want to make sure the individual map titles don't overwrite each other?

⁂

earth <- st_polygon(

x = list(
cbind(
c(rep(-180, 181), rep(180, 181), -180), c(-90:90, 90:-90, -90)
)
)
) |>
st_sfc() |>
st_set_crs(4326) |> # Equirectangular projection
st_as_sf()

gg_atm_2021 <-
ggplot() +
geom_sf(data = earth, fill = "lightblue1") +
...

This was in my notes to color the ocean blue, how do I replicate this?

That code creates a large, rectangular polygon that covers the entire global longitude (-180 to 180) and latitude (-90 to 90) grid. By placing this layer *first* in your `ggplot` chain, the countries are drawn on top of it, creating the illusion of a blue ocean background.

Since you previously decided to clip your map between 56°S (-56) and 84°N (84), you should adjust the numbers in this snippet so the ocean polygon exactly matches your clipped boundaries.

Here is how you adapt your notes for your current map:

1. Create the Clipped Ocean Polygon

The sequence `-56:84` contains 141 coordinate points (instead of the 181 points in a standard `-90:90` map). You just need to update the repetition numbers to match:

```
library(sf)

# Generate a sequence of latitudes from -56 to 84
lats <- -56:84
```

```
num_lats <- length(lats) # This equals 141

earth <- st_polygon(
    x = list(
        cbind(
            # X coordinates (Longitude: left side, then right side, then close)
            c(rep(-180, num_lats), rep(180, num_lats), -180),
            # Y coordinates (Latitude: bottom to top, then top to bottom, then close)
            c(lats, rev(lats), -56)
        )
    )
) |>
    st_sfc() |>
    st_set_crs(4326) |> # WGS84 base
    st_as_sf()
```

2. Layer it in your Map

When you build your plot, make sure `geom_sf(data = earth)` is the very first geom so it sits underneath your country polygons. Setting `color = NA` ensures it doesn't draw a harsh black border around the edge of the ocean.

```
ggplot() +
    # 1. Draw the ocean background first
    geom_sf(data = earth, fill = "lightblue1", color = NA) +

    # 2. Draw your clipped countries on top
    geom_sf(data = clipped_countries, aes(fill = gdp_per_cap)) +

    # 3. Add your continuous/discrete scales (from earlier)
    # ... your scale functions ...

    # 4. Project everything to EPSG:8857 (Equal Earth) at the end
    coord_sf(crs = "EPSG:8857")
```

When `coord_sf(crs = "EPSG:8857")` is applied, `ggplot2` will mathematically curve this rectangular ocean polygon so that it perfectly forms the beautiful, rounded edges of the Equal Earth projection.

Are you applying this to both the GDP map and the life expectancy map, or only one of them?

map_gdp <- map_gdp +

```
geom_sf(data = countries, fill = "lightblue1") +
  theme(
    legend.position = "top"
  )
```

Why does this not work, and is it possible to make it work without re-writing the entire plot?

There are two reasons why your current code snippet isn't working as expected:

1. **You used the wrong dataset:** In your snippet, you wrote `data = countries`. This means you are telling `ggplot2` to draw the shapes of the countries in light blue, rather than the `earth` (ocean) polygon we created in the previous step.
2. `ggplot2` **layers stack upwards:** When you use the `+` operator, `ggplot2` paints that new layer directly **on top** of whatever is already there. By adding the light blue layer at the end of your code, you are painting it over your GDP choropleth map, completely hiding the data underneath.

How to fix it without rewriting the plot

Normally, base layers like oceans need to be the very first `geom_sf()` in your code. However, you can use a clever `ggplot2` trick to forcefully insert a new layer at the very bottom of an existing plot's layer list without having to rebuild the whole thing.

Here is how you do it:

```
# 1. Save the ocean layer as a standalone object
# (Make sure to use 'earth' here, not 'countries'!)
ocean_layer <- geom_sf(data = earth, fill = "lightblue1", color = NA)

# 2. Inject this new layer at the very beginning (bottom) of your map's layer list
map_gdp$layers <- c(list(ocean_layer), map_gdp$layers)

# 3. Now you can safely add your theme updates normally
map_gdp <- map_gdp +
    theme(legend.position = "top")

# 4. View the result
map_gdp
```

By manually modifying `map_gdp$layers`, you slip the `lightblue1` ocean polygon securely underneath the country polygons, preserving all your hard work on the GDP colors and legends.

Did this successfully place the ocean in the background, or is it drawing a giant blue rectangle instead of a curved map?

```
# <q>You used the wrong dataset: In your snippet, you wrote data = countries</q> I ar
```

You are completely right to double-check! If you are absolutely certain that you used the correct dataset name (e.g., `earth` or whatever you named your ocean polygon), there are a few other common reasons why adding this layer might fail or throw an error:

1. CRS Mismatch

The most common issue when adding a new geometric layer is that the Coordinate Reference Systems (CRS) do not match. If your `earth` polygon was created in EPSG:4326 (standard degrees) but your `map_gdp` is already projected into EPSG:8857, `ggplot2` might struggle to reconcile them if the `coord_sf()` layer isn't set up to handle on-the-fly transformations properly.
Fix: Try projecting the ocean polygon to match your map *before* passing it to ggplot:

```
earth_projected <- st_transform(earth, 8857)
```

2. Invalid Geometries

When you manually create a bounding box or polygon (like the sequence of `-180` to `180` we generated earlier), it can sometimes create "invalid geometries" (like self-intersections at the corners) depending on the exact sequence. `ggplot2` will often silently fail to draw invalid polygons or will throw a geometry error.
Fix: Run `st_make_valid()` on your ocean polygon before plotting:

```
earth <- st_make_valid(earth)
```

3. The `inherit.aes` Argument

If your base `ggplot(countries, aes(fill = gdp_per_cap))` call defined aesthetics globally at the top level, every subsequent `geom_sf()` will try to look for a `gdp_per_cap` column in its dataset. Since your `earth` polygon doesn't have a `gdp_per_cap` column, it will fail.
Fix: Turn off aesthetic inheritance for the ocean layer:

```
geom_sf(data = earth, fill = "lightblue1", color = NA, inherit.aes = FALSE)
```

If none of these seem to be the culprit, what specific error message or visual glitch are you getting when you run your code with the correct dataset?

⁂

I do not have the st_polygon like the notes, is it possible to apply fill to the rest of the uncolored area with something like !country or the equivalent?

Yes! If you don't have an ocean polygon available, you can achieve a very similar effect simply by changing the background color of the plot panel itself using the `theme()` function.

Since `ggplot2` draws the country shapes on top of the map panel, making the panel blue will automatically color all the "empty space" (the ocean) without needing any extra spatial geometries. [207] [208]

Here is how you do it:

```
ggplot(countries) +
    geom_sf(aes(fill = gdp_per_cap)) +
    coord_sf(crs = "EPSG:8857") +
    theme(
        # Change the color of the panel background to blue
        panel.background = element_rect(fill = "lightblue1", color = NA),

        # Optional: Hide the white gridlines if you want a solid blue ocean
        panel.grid.major = element_blank()
    )
```

Important Limitations of this Method

While this is much easier than creating a custom polygon, there are two side effects to be aware of:

1. **The Square Box Effect:** A polygon is bounded exactly by its geometry, meaning if it's projected in Equal Earth, it forms a nice rounded ellipse. `panel.background`, however, colors the *entire rectangular plotting window*. This means you will get a blue box, not a blue ellipse.
2. **Clipping:** If you clip your map, the blue background will simply fill whatever rectangular area is left over after the clip.

If you don't mind the background being a solid blue square instead of a rounded ellipse, this `theme()` method is perfectly fine to use!

Would you like to know how to also change the color of the outer border of the plot (the area outside the grid)?

⁂

```
# <q>Would you like to know how to also change the color of the outer border of the p
```

To change the color of the outer border—the area surrounding your map panel where the axis labels, title, and legend sit—you use the `plot.background` argument inside the `theme()` function. [217]

Just like `panel.background`, it takes an `element_rect()` where you can define both the `fill` (the background color) and the `color` (the actual line drawn around the very edge of the image). [218] [217]

Here is how you apply it:

```
ggplot(countries) +
    geom_sf(aes(fill = gdp_per_cap)) +
    coord_sf(crs = "EPSG:8857") +
    theme(
```

```
        # 1. The inner map area (the ocean)
        panel.background = element_rect(fill = "lightblue1", color = NA),

        # 2. The outer plot area (behind the legend and titles)
        plot.background = element_rect(fill = "gray95", color = "black", size = 1)
    )
```

In this example:

- `fill = "gray95"` gives the entire image a soft, off-white background behind your text and legends.
- `color = "black"` and `size = 1` draw a solid 1pt black border around the absolute edge of your final exported image.[218]

Matching the Ocean and the Background

If you want the entire image to be one seamless blue color (so the ocean blends perfectly into the margins), you should set both backgrounds to the same color:

```
    theme(
        panel.background = element_rect(fill = "lightblue1", color = NA),
        plot.background = element_rect(fill = "lightblue1", color = NA)
    )
```

If you do this, you might also want to change `legend.background` to match so you don't get white squares behind your legend keys!

Are you trying to make the entire image a single color, or just want to differentiate the map from the surrounding white space?

⁂

I used the earth code in my notes but currently, the side of the plot is being cut out, probably due to our clipping, how do I ensure that it doesnt get clipped?

The issue you are experiencing happens because `st_crop()` physically cuts the polygons, including your `earth` polygon. If your `earth` polygon does not extend past the plot panel limits, `ggplot2` will sometimes draw the borders of that cropped polygon right on the edge of the screen, creating a visible vertical "cut" line on the sides where the map ends.[227] [228]

There are a few ways to ensure the sides of your map don't look awkwardly cut off:

Method 1: Turn off Clipping in `coord_sf()`

By default, `ggplot2` strictly clips (hides) any drawing that goes outside the plotting panel. You can tell `coord_sf()` to let the polygons render slightly outside the boundaries by setting `clip = "off"`. This often fixes the harsh lines on the edge of the plot.[227]

```
map_gdp +
    coord_sf(crs = "EPSG:8857", clip = "off")
```

Method 2: Crop the Earth Polygon Slightly Larger

If the first method doesn't work, the problem might be that `st_crop()` cut your `earth` polygon exactly at the boundary of your `countries` polygons.

Instead of cropping `earth` to `-180` and `180` longitude, make the `earth` polygon slightly wider than the data you are plotting. Since longitude mathematically stops at `180` and `-180`, you can create the `earth` polygon using those limits, but when you crop your **countries**, crop them slightly *smaller* (e.g., `-179` to `179`). This gives the ocean a tiny bit of "bleed" room on the sides so the border line is pushed out of the visible window.

Method 3: Remove the Ocean Border entirely

If the issue is simply that you can see a dark line running vertically down the sides of the ocean, make sure you explicitly set `color = NA` (not just `fill`) inside the `geom_sf()` layer for the earth. This removes the stroke outline of the polygon.[229]

```
geom_sf(data = earth, fill = "lightblue1", color = NA)
```

Are you seeing a dark line on the sides, or is the actual light blue fill color not reaching the edge of the white background panel?

⁂

We tried to expand the xlim and exclude expand=FALSE, why did it not work?

When using `coord_sf()`, the `expand = FALSE` argument and the `xlim` limits interact in a very specific, sometimes frustrating way due to how `ggplot2` handles spatial projections.[238] [239]

If you tried expanding the `xlim` and removing `expand = FALSE`, and your map still didn't look right, here is why:

1. Zero-expansion Errors

When `expand = FALSE` is used, `ggplot2` completely disables the tiny padding it normally puts between your data and the edge of the plot frame. In spatial data, if a geometry happens to land exactly on the mathematically perfect edge of a coordinate grid, removing that padding can cause internal calculation errors (like mismatched break labels) or cause polygons to stretch out of bounds. [240]

2. Limits Need to be in Projected Units

If you set `xlim` but left out `expand = FALSE`, your limits might have simply been ignored or rendered as a tiny dot if they were in the wrong units. Remember, because your map uses `crs = "EPSG:8857"`, the X and Y coordinates are mathematically calculated in **meters**, not degrees. [241]

If you wrote something like `xlim = c(-180, 180)` on an Equal Earth projection, `ggplot2` tried to zoom into a microscopic 360-meter wide box in the ocean. The true width of an Equal Earth map is tens of millions of meters.

The Best Workaround

If you want to manually adjust the horizontal limits of your map to prevent clipping without using `expand = FALSE`, you can use the `expansion()` function directly inside the scale, rather than `coord_sf()`.

```
map_gdp +
    coord_sf(crs = "EPSG:8857") +
    scale_x_continuous(expand = expansion(mult = 0.05)) # Adjust this multiplier
```

This keeps the default coordinate behavior but slightly increases (or decreases) the padding around the map, which is often enough to push the ocean boundaries safely off-screen so you don't see the dark vertical cut lines.

Are you still trying to get the ocean layer to perfectly meet the edge of the panel without drawing a border?

⁂

I ran into an error using shadowtext,

```
geom_shadowtext(
    data = top_40_life,
    aes(label = code, geometry = geometry),
    bg.color = "white",
    color = "black",
    check_overlap = TRUE
  )
```

Error in `geom_shadowtext()`:
! Problem while setting up geom.
i Error occurred in the 5th layer.
Caused by error in `compute_geom_1()`:
! `geom_shadow_text()` requires the following missing aesthetics: x and
y.

The error occurs because `geom_shadowtext()` from the `shadowtext` package does not have native support for `sf` simple feature geometries (the `geometry` aesthetic) like `geom_sf_text()` does.[253]

Because it is a regular text geom, it strictly requires explicit `x` and `y` coordinates for every point, whereas `geom_sf_text()` automatically calculates those coordinates from your polygon centroids internally.[253]

To use `geom_shadowtext()` with spatial data, you must extract the X and Y coordinates (longitude and latitude) from your polygons first and pass them directly into the `aes()`.

How to Fix It

You can calculate the center points (centroids) of your countries and extract their `x` and `y` coordinates using `st_coordinates()` and `st_centroid()`.

```
library(shadowtext)

# 1. Calculate the centroids of your top_40_life polygons
# st_centroid gets the center, st_coordinates pulls out the raw X and Y
centroids <- st_coordinates(st_centroid(top_40_life$geometry))

# 2. Add these coordinates back into your dataset as new columns
top_40_life$x <- centroids[, 1]  # Longitude / X
top_40_life$y <- centroids[, 2]  # Latitude / Y

# 3. Now use geom_shadowtext mapping to these new columns
geom_shadowtext(
    data = top_40_life,
    aes(x = x, y = y, label = code), # Pass x and y explicitly here
    bg.color = "white",
    color = "black",
    check_overlap = TRUE
)
```

A quick tip: If your `top_40_life` map is being plotted in your `EPSG:8857` projection, make sure your centroids are calculated *after* the data is transformed to that CRS, or `ggplot2` might place the text in the wrong spots.

Did extracting the centroids put the labels in the correct locations on your map?

⁂

that did not work fully just yet. Can you help me check if my x and y points are done correctly before I use geom_shadowtxt

To check if your points were correctly generated before you run `geom_shadowtext()`, you can quickly use base R's standard `head()` function to inspect the dataframe or use `ggplot2` to draw the points on a temporary map.

Here is how you can verify your `x` and `y` coordinates:

1. Inspect the Dataframe

Run this in your console to look at the first few rows of your `top_40_life` dataset:

```
head(top_40_life %>% select(code, x, y))
```

If the extraction worked, you should see three columns: the country `code`, followed by the `x` (longitude values, usually between -180 and 180) and `y` (latitude values, usually between -90 and 90). If `x` and `y` are entirely missing or look wildly incorrect (like giant numbers in the millions), the centroid extraction failed or grabbed the wrong geometry list.

2. Visualize the Points

The best way to know if they are exactly where they are supposed to be is to plot them as simple dots using `geom_point()`. Run this simple plot:

```
ggplot() +
    # Draw your base map
    geom_sf(data = clipped_countries) +

    # Draw your extracted coordinates as red dots
    geom_point(data = top_40_life, aes(x = x, y = y), color = "red", size = 2) +

    coord_sf(crs = "EPSG:8857")
```

If you see red dots appearing squarely in the middle of your 40 countries, your coordinates are perfectly correct, and `geom_shadowtext()` will place the labels in those exact spots!

If the dots are clustered in the ocean, squished at the 0,0 line, or throwing an error, that tells you the coordinates were not generated correctly.

Are the `x` and `y` values showing up when you run `head()`, or did it return an error?

⁂

Currently my visualization is alot smaller compared to my legends, this is a snippet of my current plots,

```
map_gdp <- ggplot(countries) +
  geom_sf(data = earth, fill = "aliceblue") +
  geom_sf(
    aes(fill = gdp_per_cap),
    color = "black",
    linewidth = 0.1
  ) +
  scale_fill_fermenter(
    name = "GDP per capita ($ Purchasing Power Parity 2021)",
    palette = "Reds",
    direction = 1,
    breaks = c(2000, 5000, 10000, 20000, 50000, 100000),
    labels = label_number(scale_cut = cut_short_scale()),
    guide = guide_colorsteps(position = "top")
  ) +
  geom_sf(
    data = filter(countries, is.na(gdp_per_cap)),
    aes(color = "Missing"),
    fill = "lightgray",
    linewidth = 0.1
  ) +
  geom_sf(
    data = filter(countries, status == "Non-determined legal status area"),
    aes(color = "Undetermined"),
    fill = "darkgray",
    linewidth = 0.1
  ) +
  scale_color_manual(
    name = "",
    values = c("Missing" = "black", "Undetermined" = "black"),
    guide = guide_legend(
      position = "bottom",
      override.aes = list(
        fill = c("Missing" = "lightgray", "Undetermined" = "darkgray"),
        color = "black",
        linewidth = 0.1
      )
    )
  ) +
  geom_shadowtext(
    data = top_40_gdp,
    aes(label = code, geometry = geometry),
```

```
    stat = "sf_coordinates",
    bg.color = "white",
    color = "black",
  ) +
  theme_void() +
  theme(
    legend.margin = margin(5, 0, 0, 0),
    legend.frame = element_rect(),
  ) +
  coord_sf(
    crs = "EPSG:8857",
    ylim = c(-56, 84),
    xlim = c(-179.99, 179.99),
    default_crs = 4326,
    expand = FALSE
  )

==========

map_life_exp / map_gdp +
  plot_annotation(
    title = "Comparing wealth and longevity across nations (2024)",
    caption = "Undetermined refers to an area's legal source \nSource: World Bank"
  )
```

How can I revise this to choose suitable dimensions?

It looks like your plots are being squeezed because the large legends in both plots are taking up a significant amount of vertical and horizontal space. Since `patchwork` tries to cram everything (two maps and two sets of legends) into your viewing window, the maps shrink to fit.

Here is how you can revise your code to fix the dimensions and clean up the layout:

1. Collect and Position the Legends using Patchwork

Right now, each map is generating its own legends, which takes up a lot of room. Since `patchwork` allows you to control guides globally, you can tell it to pull all the legends out of the individual plots and group them together in a designated area.

Add `plot_layout(guides = "collect")` to your final patchwork call, and use the `&` operator to apply a global theme to push those collected legends to the bottom of the entire image (or the side).[273]

```
map_life_exp / map_gdp +
    plot_annotation(
        title = "Comparing wealth and longevity across nations (2024)",
        caption = "Undetermined refers to an area's legal source \nSource: World Bank
```

```
    ) +
    # Collect all legends from the plots
    plot_layout(guides = "collect") &
    # Force the collected legends to sit at the bottom of the entire image
    theme(legend.position = "bottom")
```

2. Adjust Guide Sizes in Your Base Maps

Your continuous scale (`guide_colorsteps`) is probably drawing a very thick color bar, which steals vertical space from the maps. You can manually adjust the width and height of the color bars directly inside your `guide_colorsteps()` function in both `map_gdp` and `map_life_exp`.

Change your GDP scale guide to something like this:

```
scale_fill_fermenter(
    name = "GDP per capita ($ Purchasing Power Parity 2021)",
    palette = "Reds",
    direction = 1,
    breaks = c(2000, 5000, 10000, 20000, 50000, 100000),
    labels = label_number(scale_cut = cut_short_scale()),
    guide = guide_colorsteps(
        barwidth = 15,    # Make it wider horizontally
        barheight = 0.5,  # Make it much thinner vertically
        title.position = "top", # Put the title above the bar to save horizontal spac
        title.hjust = 0.5
    )
)
```

3. Adjust Export Dimensions

If you are viewing this in the standard RStudio plotting window, it will always look a bit squished because the default window isn't very tall. When you are ready to save the final image using `ggsave()`, manually set the dimensions to be taller so the maps have room to breathe:

```
ggsave("combined_maps.png", width = 10, height = 12, dpi = 300)
```

Does collecting the legends into the bottom panel give the maps enough room to expand, or do they still look distorted?

⁂

This was how I defined my top 40 countries,

top_40_gdp <- countries |>
  slice_max(order_by = gdp_per_cap, n = 40)

top_40_life <- countries |>
  slice_max(order_by = life_exp, n = 40)

"Country labels: Add the codes of the 40 largest countries as labels to the map withoutexcessive overlap. Give each label a thin white halo with the shadowtext package so thatit remains legible on any fill."
Is this inline with what the aim we are trying to achieve?

Based on your instructions, your current code is **not** quite in line with the goal.

The prompt says: *"Add the codes of the 40 **largest countries** as labels to the map."*

In your current code, you are using `slice_max()` to find the top 40 countries by **GDP** (`order_by = gdp_per_cap`) and by **Life Expectancy** (`order_by = life_exp`). This will grab the richest and longest-living countries (like Luxembourg or Monaco), which are often geographically very small.

The instruction specifically asks for the "largest" countries, which in a mapping context means largest by **physical land area**. You want the largest physical countries (like Russia, Canada, USA, China, Brazil) because those are the polygons that are big enough to comfortably fit text labels without creating a massive, overlapping mess on the map. [283] [284]

How to fix it

You need to create a single dataset containing the 40 largest countries by area.

If your `countries` dataset already has a column for land area (e.g., `area_km2`, `area`, `shape_area`), you should do this:

```
# Find the 40 largest countries by land area
top_40_largest <- countries |>
    slice_max(order_by = area_km2, n = 40) # Change area_km2 to match your column nar

# Extract their centroids (as we discussed previously)
centroids <- st_coordinates(st_centroid(top_40_largest$geometry))
top_40_largest$x <- centroids[, 1]
top_40_largest$y <- centroids[, 2]
```

Adding the Labels

Then, instead of using `top_40_gdp` and `top_40_life` separately on each map, you will use this single `top_40_largest` dataset for the `geom_shadowtext()` layer on **both** maps.

```
geom_shadowtext(
    data = top_40_largest,
    aes(x = x, y = y, label = code),
    bg.color = "white",
    color = "black",
    check_overlap = TRUE
)
```

(Remember to remove `stat = "sf_coordinates"` *and* `geometry = geometry` *from your previous* `geom_shadowtext` *call, since you are manually passing* `x` *and* `y` *now).*

Using the 40 physically largest countries ensures the labels fall cleanly on big landmasses, making both the GDP and life expectancy colors underneath clearly visible.

Do you have a column for land area in your dataset, or do you need to calculate the area using the geometries first?

⁂

Upon further checks, there is no such value as pop, only gdp_per_cap and life_exp

No problem! If your dataset doesn't already have an area column, `sf` makes it incredibly easy to calculate the exact land area for every country on the fly using its geometry.

You can use the `st_area()` function to compute the area of each polygon. [294] [295]

Here is how you can calculate the area, add it to your dataset, and extract the 40 physically largest countries:

```
library(sf)
library(dplyr)

# 1. Calculate the area for each country geometry
# (sf will automatically calculate this in square meters if using a projected CRS)
countries$area <- st_area(countries$geometry)

# 2. Extract the 40 largest countries based on that new area column
top_40_largest <- countries |>
    slice_max(order_by = area, n = 40)

# 3. Calculate the centroids for these 40 countries to get X and Y coordinates
centroids <- st_coordinates(st_centroid(top_40_largest$geometry))

top_40_largest$x <- centroids[, 1]
top_40_largest$y <- centroids[, 2]
```

Once you run this code, your `top_40_largest` object will contain the 40 biggest landmasses in the world (like Russia, Canada, the USA, and China).

You can then pass this exact dataset into your `geom_shadowtext()` layer for both the GDP map and the life expectancy map as we discussed previously!

Are there any warnings when running `st_centroid()`, such as "st_centroid assumes attributes are constant over geometries of x"?

⁂

As we used patchwork to perform our labels instead, how do we centralize the text and apply a bold to the title?

This is our current code:
map_life_exp / map_gdp +
  plot_annotation(
    title = "Comparing wealth and longevity across nations (2024)",
    caption = "Undetermined refers to an area's legal source \nSource: World Bank"
)

To centralize the title and make it bold when using `patchwork`'s `plot_annotation()`, you need to use its `theme` argument.

Because `plot_annotation()` generates a completely separate set of global elements that wrap around your plots, it does not inherit the specific theme settings from your individual maps. You have to explicitly pass an `element_text()` instruction directly to it. [304] [305]

Here is how you revise your code:

```
map_life_exp / map_gdp +
    plot_annotation(
        title = "Comparing wealth and longevity across nations (2024)",
        caption = "Undetermined refers to an area's legal source \nSource: World Banl
        # Pass a specific theme to format the global title
        theme = theme(
            plot.title = element_text(
                hjust = 0.5,      # This centers the text
                face = "bold",    # This makes it bold
                size = 16         # Optional: Make it slightly larger
            )
        )
    ) +
    # If you used guides = "collect" earlier, make sure you append it using &
    plot_layout(guides = "collect") &
    theme(legend.position = "bottom")
```

The `hjust = 0.5` mathematically centers the title exactly in the middle of the entire `patchwork` canvas. [306] [305]

Are you happy with the alignment of your World Bank caption as well, or do you want to move that to a specific corner?

⁂

I added "legend.frame = element_rect()" to draw a outline around the color bar, however, I would like for the color palette bar to be bolder

```
theme(
    legend.margin = margin(5, 0, 0, 0),
    legend.frame = element_rect(),
    legend.position = "top"
)
```

If you want to modify the thickness or visual boldness of the frame you just added around the overall legend, you can customize the `element_rect()` inside `legend.frame`.[314]

By default, an empty `element_rect()` simply draws a standard thin black line. You can make it bolder by adjusting the `linewidth` (or `size` in older versions of `ggplot2`) and the `color` arguments.

Here is how you can update your code:

```
theme(
    legend.margin = margin(5, 0, 0, 0),
    legend.position = "top",
    # Make the frame line much thicker and distinctly black
    legend.frame = element_rect(color = "black", linewidth = 1.5)
)
```

Are you trying to border the legend box or the color scale itself?

There is sometimes a bit of confusion in `ggplot2` about the different boxes around legends:

1. `legend.frame` / `legend.box.background`: This draws a border around the *entire* legend area (the title, the text, and the colors combined).
2. `frame.colour` **and** `frame.linewidth` **in** `guide_colorsteps()`: If your goal is specifically to draw a bold outline strictly around the red colored blocks of your GDP scale (the colorbar itself), you must do that inside your `scale_fill_fermenter()` function, not in the `theme()`.[315]

If you want the outline around just the colored blocks to be bolder, you would write:

```
scale_fill_fermenter(
    # ... your other arguments ...
    guide = guide_colorsteps(
        position = "top",
        frame.colour = "black",   # Adds a black border to the colorbar
        frame.linewidth = 1.5     # Makes that border thick
    )
)
```

Are you looking to thicken the outline of the entire legend box, or just the colorbar itself?

⁂

Currently this is what I have written in terms of what conclusions we can draw from this graph, what do you think?

"By stacking the Life Expectancy map directly above the GDP per Capita map, the user is able to see and compare the Wealth to Longetivity correlation. For example, countries like Australia mirror the same dark color band for both GDP per capita and Life expectancy.

They are also able to observe certain outliers where the life expectancy do not align with the economic wealth, e.g. while USA is in the dark band of GDP per capita, it is in the slightly lighter green band for life expectancy."

Your conclusions are excellent and accurately capture the primary value of visualizing these two maps together. You have successfully identified both the overarching geographical trend and a highly relevant real-world outlier.

Here is a quick review of your text and a few suggestions to make it even stronger:

Strengths of Your Analysis

Your draft does a fantastic job of explaining *why* the layout matters by explicitly stating that the stacked design facilitates direct spatial comparison. Using Australia as a baseline example of the positive correlation grounds the reader, making your transition into the United States outlier much more impactful and easier to understand.

Minor Corrections

There are only two tiny mechanical errors to fix before you publish. "Longetivity" should be spelled "longevity", and "life expectancy do not align" should be "life expectancy does not align" since life expectancy is a singular noun.

Suggested Polish

If you want to make the text sound slightly more academic or professional, you can tighten the phrasing to focus on the data rather than "the user". Here is a slightly polished version of your exact thoughts:

> "Stacking the Life Expectancy map directly above the GDP per Capita map allows viewers to easily observe the correlation between national wealth and longevity. For example, countries like Australia exhibit the darkest color bands on both maps, reflecting this positive relationship. However, the visualization also highlights notable outliers where life expectancy does not strictly align with economic wealth; the USA, despite sitting in the highest tier for GDP per capita, falls into a lower tier for life expectancy."

Are you planning to add this text directly onto the plot as a caption, or will it be written in the body of your report?

1. https://www.youtube.com/watch?v=tvm4bJf1xzI
2. https://positron.posit.co/welcome.html
3. https://positron.posit.co/guide-r.html
4. https://posit.co/blog/eda-in-positron
5. https://cloud.r-project.org/web/packages/positron.tutorials/positron.tutorials.pdf
6. https://ppbds.github.io/positron.tutorials/
7. https://www.youtube.com/watch?v=4Ir_HX4riHw
8. https://www.youtube.com/watch?v=VPHuCl8asjQ
9. https://www.youtube.com/watch?v=ZkZXrJNepP0
10. https://www.youtube.com/watch?v=NVKJK0aOSkI
11. https://positron.posit.co/tutorial-get-started-quarto.html
12. https://www.youtube.com/watch?v=mru9z50IOhI
13. https://www.youtube.com/watch?v=ndq2Mm3Dju8
14. https://www.youtube.com/watch?v=-jhTjOK4vQ8
15. https://www.youtube.com/watch?v=SBPVcKbFbAk
16. https://r-spatial.github.io/sf/reference/valid.html
17. https://stackoverflow.com/questions/70163367/sf-in-r-one-projection-works-another-throws-an-invalid-geometry-error
18. https://r-spatial.org/r/2017/03/19/invalid.html
19. https://www.rdocumentation.org/packages/sf/versions/1.0-23/topics/valid
20. https://stackoverflow.com/questions/68347225/invalid-geometry-using-´sf´-in-r
21. https://github.com/r-spatial/sf/issues/174
22. http://r-spatial.github.io/sf/reference/valid.html
23. https://r-spatial.r-universe.dev/sf/doc/manual.html
24. https://github.com/r-spatial/sf/issues/1732
25. https://github.com/r-spatial/s2/issues/99
26. https://www.rdocumentation.org/packages/sf/versions/1.0-20/topics/st_cast
27. https://github.com/r-spatial/sf/issues/763
28. https://search.r-project.org/CRAN/refmans/sf/html/st_cast.html
29. https://stackoverflow.com/questions/63688805/r-how-to-merge-multipolygons-to-polygon
30. https://stackoverflow.com/questions/75936092/how-to-change-multipolygon-to-polygon-in-r
31. https://github.com/r-spatial/sf/issues/1883
32. https://github.com/r-spatial/sf/blob/main/R/cast_sfg.R
33. https://github.com/r-spatial/sf/issues/461
34. https://rdrr.io/cran/sf/src/R/cast_sfg.R

35. https://r.geocompx.org/geometry-operations
36. https://www.rdocumentation.org/packages/rmapshaper/versions/0.6.0/topics/ms_simplify
37. https://tmap.geocompx.org/xx-data_processing
38. https://seasmith.github.io/blog/simplify_your_map_to_reduce_drawing_time/
39. https://cran.r-project.org/web/packages/rmapshaper/rmapshaper.pdf
40. https://www.carlos-toruno.com/blog/geosimplify/
41. https://www.rdocumentation.org/packages/rmapshaper/versions/0.1.0/topics/ms_simplify
42. https://centerline.anatolii.nz/articles/simplify
43. https://stackoverflow.com/questions/62034446/simplify-my-us-state-shapefile-with-rmapshaperms-simplify-gives-error
44. https://www.r-bloggers.com/2021/03/simplifying-geospatial-features-in-r-with-sf-and-rmapshaper/
45. https://r.geocompx.org/solutions/geometric-operations.html
46. https://bin-ye.com/post/2019/06/30/geocomputation-with-r-05/
47. https://andyteucher.ca/rmapshaper/
48. https://cran.r-project.org/web/packages/centerline/refman/centerline.html
49. https://rdrr.io/cran/tmaptools/man/simplify_shape.html
50. https://www.youtube.com/watch?v=0b6hIrcSzYk
51. https://cran.r-project.org/web/packages/rmapshaper/vignettes/rmapshaper.html
52. https://r-graph-gallery.com/169-170-basic-manipulation-of-shapefiles.html
53. https://mapshaper.org/docs/guides/simplification.html
54. https://tmap.geocompx.org/data-processing
55. https://github.com/mbloch/mapshaper/issues/80
56. https://www.r-bloggers.com/2021/03/simplifying-geospatial-features-in-r-with-sf-and-rmapshaper/
57. https://stackoverflow.com/questions/48360446/fit-two-sf-polygons-seamlessly
58. https://bin-ye.com/post/2019/06/30/geocomputation-with-r-05/
59. https://www.carlos-toruno.com/blog/geosimplify/
60. https://github.com/r-spatial/sf/issues/381
61. http://www.vis4.net/blog/simplification-of-country-outlines/
62. https://www.rdocumentation.org/packages/rmapshaper/versions/0.6.0/topics/ms_simplify
63. https://www.rdocumentation.org/packages/rmapshaper/versions/0.4.0/topics/ms_simplify
64. https://stackoverflow.com/questions/79598830/efficient-handling-of-shapefiles-with-many-polygons-in-r-simplification-and-mer
65. https://seasmith.github.io/blog/simplify_your_map_to_reduce_drawing_time/
66. https://r-spatial.github.io/sf/reference/st_transform.html
67. https://stackoverflow.com/questions/73716269/visualizing-buffers-that-cross-the-dateline
68. https://github.com/r-spatial/sf/issues/674
69. https://stackoverflow.com/questions/73098840/polygons-are-not-splitting-across-the-international-dateline-as-expected-in-r-us

70. https://github.com/r-spatial/sf/issues/541
71. https://github.com/r-spatial/sf/issues/1790
72. https://www.rdocumentation.org/packages/sf/versions/0.4-2/topics/st_transform
73. https://github.com/r-spatial/sf/issues/280
74. https://search.r-project.org/CRAN/refmans/ggspatial/html/geom_spatial_segment.html
75. https://github.com/r-spatial/sf/issues/1999
76. https://github.com/r-spatial/sf/issues/2250
77. https://stackoverflow.com/questions/71641417/how-to-create-a-polygon-combine-polygons-that-cross-the-180-meridian-dateline
78. https://github.com/r-spatial/sf/issues/1790
79. https://github.com/r-spatial/sf/issues/541
80. https://cran.rstudio.com/web/packages/sf/sf.pdf
81. https://stackoverflow.com/questions/73098840/polygons-are-not-splitting-across-the-international-dateline-as-expected-in-r-us
82. https://r-spatial.github.io/sf/reference/st_transform.html
83. https://github.com/r-spatial/sf/issues/1862
84. https://www.rdocumentation.org/packages/sf/versions/0.5-4/topics/st_transform
85. https://github.com/r-spatial/sf/issues/1999
86. https://github.com/r-spatial/sf/issues/669
87. https://www.r-bloggers.com/2023/07/finding-duplicate-values-in-a-data-frame-in-r-a-guide-using-base-r-and-dplyr/
88. https://stackoverflow.com/questions/68141127/how-to-avoid-duplicate-a-code-for-one-output
89. https://github.com/dmbwebb/dups
90. https://stackoverflow.com/questions/76710442/how-to-arrange-duplicated-sample-codes-to-follow-on-another-in-an-output
91. https://www.stata.com/support/faqs/data-management/duplicate-observations/
92. https://rdrr.io/cran/spatialEco/man/remove_duplicates.html
93. https://stat.ethz.ch/R-manual/R-devel/library/base/html/duplicated.html
94. https://refactoring.guru/smells/duplicate-code
95. https://www.datanovia.com/en/lessons/identify-and-remove-duplicate-data-in-r/
96. https://github.com/r-spatial/sf/issues/49
97. https://github.com/r-spatial/sf/issues/798
98. https://stackoverflow.com/questions/58649456/why-does-rbind-work-and-bind-rows-not-work-in-combining-these-sf-objects
99. https://forum.posit.co/t/performing-a-full-join-on-sf-objects/43902
100. https://www.rdocumentation.org/packages/sf/versions/1.0-22/topics/bind
101. https://rdrr.io/cran/sf/man/bind.html
102. https://r-spatial.github.io/sf/reference/bind.html
103. https://search.r-project.org/CRAN/refmans/tidyterra/html/bind_rows.SpatVector.html

104. https://stackoverflow.com/questions/51312935/convert-a-list-of-sf-objects-into-one-sf/51313522
105. https://www.rdocumentation.org/packages/tidyterra/versions/1.0.0/topics/bind_rows.SpatVector
106. https://stackoverflow.com/questions/79058429/how-to-correct-erroneous-crs-in-the-function-coord-sf-in-ggplot-after-merging
107. https://epsg.io/8857
108. https://www.rdocumentation.org/packages/ggplot2/versions/3.1.1/topics/ggsf
109. https://stackoverflow.com/questions/72832304/ggplot-coord-sf-breaks-map
110. https://github.com/tidyverse/ggplot2/issues/2857
111. https://ggplot2.tidyverse.org/reference/ggsf.html
112. https://github.com/tidyverse/ggplot2/issues/6382
113. https://github.com/r-spatial/sf/issues/1645
114. https://github.com/tidyverse/ggplot2/blob/main/R/geom-sf.R
115. https://github.com/tidyverse/ggplot2/issues/4494
116. https://github.com/tidyverse/ggplot2/issues/3944
117. https://cran.r-project.org/web/packages/sfhelper/readme/README.html
118. https://stackoverflow.com/questions/68343004/ggplot-and-geom-sf-not-showing-graticule-and-tick-marks-on-world-map
119. https://r-graph-gallery.com/package/sf.html
120. https://stackoverflow.com/questions/44887635/plotting-a-sf-object-with-geom-sf-with-any-projection-other-than-lat-long
121. image.jpg
122. https://gist.github.com/marcosci/ce0d7739a11a77a930c7c2072d0c74ee
123. https://www.r-bloggers.com/2019/04/zooming-in-on-maps-with-sf-and-ggplot2/
124. https://ggplot2.tidyverse.org/reference/ggsf.html
125. https://forum.posit.co/t/coord-sf-collapses-all-points-to-0-0/25814
126. https://github.com/r-spatial/sf/issues/396
127. https://ggplot2.tidyverse.org/reference/coord_map.html
128. https://stackoverflow.com/questions/78662643/map-projections-using-ggplot
129. https://github.com/tidyverse/ggplot2/issues/2985
130. https://stackoverflow.com/questions/72832304/ggplot-coord-sf-breaks-map
131. https://stackoverflow.com/questions/78599581/how-to-set-transparent-plot-background-but-fill-map-panel-for-mollweide-or-robin
132. https://github.com/r-spatial/sf/issues/1361
133. https://ggplot2.tidyverse.org/reference/ggsf.html
134. https://stackoverflow.com/questions/65565070/geom-sf-fill-fails-when-xlim-ylim-are-specified-using-coord-sf-looking-for-a-w
135. https://stackoverflow.com/questions/66031935/ggplot2-and-sf-geom-sf-text-within-limits-set-by-coord-sf/66032492
136. https://forum.generic-mapping-tools.org/t/trouble-with-mollweide-projection/5829

137. https://github.com/tidyverse/ggplot2/issues/3944
138. https://datascience.blog.wzb.eu/2019/04/30/zooming-in-on-maps-with-sf-and-ggplot2/
139. https://gist.github.com/marcosci/ce0d7739a11a77a930c7c2072d0c74ee
140. https://forum.posit.co/t/coord-sf-collapses-all-points-to-0-0/25814
141. https://stackoverflow.com/questions/70724669/why-do-some-xlims-and-ylims-produce-this-error-in-ggplot-and-sf
142. https://ggplot2.tidyverse.org/reference/scale_brewer.html
143. https://rdrr.io/cran/colorspace/man/scale_colour_continuous_sequential.html
144. https://www.nceas.ucsb.edu/sites/default/files/2020-04/colorPaletteCheatsheet.pdf
145. https://colorspace.r-forge.r-project.org/articles/ggplot2_color_scales.html
146. http://r-statistics.co/R-Color-Theory-ggplot2.html
147. https://z3tt.github.io/ukraine_colors_workshop/slides
148. https://www.datanovia.com/en/blog/top-r-color-palettes-to-know-for-great-data-visualization/
149. https://www.rdocumentation.org/packages/colorspace/versions/2.1-2/topics/scale_colour_continuous_sequential
150. https://campus.datacamp.com/courses/introduction-to-data-visualization-with-ggplot2/geometries?ex=12
151. https://cran.r-project.org/web/packages/ggsci/vignettes/ggsci.html
152. https://dcl-2019-01.github.io/curriculum/spatial-vis.html
153. https://stackoverflow.com/questions/66958367/ggplot2-and-sf-coord-fixed-doesnt-work-combined-with-geom-sf-and-coord-sf
154. https://nrennie.rbind.io/blog/mapping-a-marathon-with-rstrava/
155. https://upgo.lab.mcgill.ca/2019/12/13/making-beautiful-maps/
156. https://edzer.github.io/sdsr_exercises/13.html
157. https://gist.github.com/ateucher/1467691edbc1fd1f7fbbabd05957cbb5
158. https://stackoverflow.com/questions/69832290/st-contour-mapping-should-be-created-with-aes
159. https://stackoverflow.com/questions/74733975/setting-crs-for-plotting-with-stamenmaps
160. https://plotly.com/ggplot2/layers/geoms/geom_sf/
161. https://stackoverflow.com/questions/78711332/adding-a-geom-sf-to-a-long-lat-plot-with-gratia-package-in-r
162. https://stackoverflow.com/questions/17921961/how-to-clip-a-coord-polar-plot
163. https://dave-clark.github.io/post/making-maps-with-ggplot2-and-sf/
164. https://stackoverflow.com/questions/77977780/cropping-a-map-and-preserving-coordinate-points-in-r
165. https://ggplot2.tidyverse.org/reference/ggsf.html
166. https://ggplot2.tidyverse.org/reference/coord_map.html
167. https://ggplot2-book.org/coord.html
168. https://stackoverflow.com/questions/68666805/clipping-ggplot2-map-by-longitude-r
169. https://ggplot2-book.org/maps.html
170. https://www.r-bloggers.com/2019/04/zooming-in-on-maps-with-sf-and-ggplot2/
171. https://prism.dev.a2-ai.cloud/docs/ggplot2/4.0.1/reference/coord_map.html
172. https://www.rdocumentation.org/packages/ggplot2/versions/3.3.2/topics/CoordSf

173. https://rdrr.io/github/hadley/ggplot2/src/R/coord-sf.R
174. https://ggplot2.tidyverse.org/reference/ggsf.html
175. https://github.com/tidyverse/ggplot2/blob/main/R/coord-map.R
176. https://r-spatial.org/r/2018/10/25/ggplot2-sf-2.html
177. https://stackoverflow.com/questions/66958367/ggplot2-and-sf-coord-fixed-doesnt-work-combined-with-geom-sf-and-coord-sf
178. https://cidm-ph.github.io/ggmapinset/reference/coord_sf_inset.html
179. https://github.com/r-spatial/sf/issues/2348
180. https://ggplot2.tidyverse.org/reference/index.html
181. https://r-spatial.org/r/2018/10/25/ggplot2-sf.html
182. https://github.com/tidyverse/ggplot2/blob/main/R/geom-sf.R
183. https://ggplot2-book.org/maps.html
184. https://github.com/tidyverse/ggplot2/issues/2938
185. https://github.com/tidyverse/ggplot2/issues/4494
186. https://ggplot2.tidyverse.org/reference/coord_map.html
187. https://patchwork.data-imaginist.com/articles/guides/annotation.html
188. https://github.com/thomasp85/patchwork/issues/43
189. https://stackoverflow.com/questions/67785239/labels-ggplot-patchwork
190. https://stackoverflow.com/questions/75984354/putting-a-common-label-for-two-r-base-graphics-subplots-within-a-figure-genera
191. https://stackoverflow.com/questions/17576381/label-individual-panels-in-a-multi-panel-ggplot2
192. https://cran.r-project.org/web/packages/patchwork/patchwork.pdf
193. https://stackoverflow.com/questions/69065003/using-patchwork-to-annotate-plots-that-have-labels-in-different-positions
194. https://forum.posit.co/t/how-to-tag-some-plots/73392
195. https://github.com/thomasp85/patchwork/issues/1
196. https://www.youtube.com/watch?v=N1wlhTCEjr4
197. https://www.stata.com/manuals13/perror.pdf
198. https://statmodeling.stat.columbia.edu/2013/04/17/data-problems-coding-errors-what-can-be-done/
199. https://www.youtube.com/watch?v=8Gpms4dV9D0
200. https://statsandr.com/blog/top-10-errors-in-r/
201. https://365datascience.com/question/dataset-please-kindly-confirm-this/
202. https://devflowstack.org/blog/dataset-structure-mismatch-between-the
203. https://www.geeksforgeeks.org/data-science/which-is-correct-dataset-or-data-set/
204. https://developer.mozilla.org/en-US/docs/Web/API/HTMLElement/dataset
205. https://stackoverflow.com/questions/28007453/why-when-i-want-to-create-a-new-dataset-in-code-below-it-works-wrong-when-i-use
206. https://ies.ed.gov/rel-central/2025/01/common-sources-data-errors-and-error-checking-techniques

207. https://r-charts.com/ggplot2/background-color/
208. https://miriamlerma.netlify.app/posts/2023-03-04-mapping-in-r/
209. https://r-spatial.org/r/2018/10/25/ggplot2-sf.html
210. https://ggplot2.tidyverse.org/articles/faq-customising.html
211. https://stackoverflow.com/questions/6736378/how-do-i-change-the-background-color-of-a-plot-made-with-ggplot2
212. https://www.r-bloggers.com/2022/07/how-to-change-background-color-in-ggplot2-3/
213. https://www.tutorialspoint.com/ggplot2/ggplot2_background_colors.htm
214. https://www.sthda.com/english/wiki/ggplot2-themes-and-background-colors-the-3-elements
215. https://www.geeksforgeeks.org/r-language/how-to-change-background-color-in-r-using-ggplot2/
216. https://stackoverflow.com/questions/52781189/ggplot2-change-background-color-of-panel-and-plot-independently
217. https://r-graph-gallery.com/240-custom-layout-background-ggplot2.html
218. https://r-charts.com/ggplot2/background-color/
219. https://bookdown.dongzhuoer.com/hadley/ggplot2-book/modifying-theme-components
220. https://ggplot2.tidyverse.org/reference/theme.html
221. https://ggplot2-book.org/themes.html
222. https://stackoverflow.com/questions/6736378/how-do-i-change-the-background-color-of-a-plot-made-with-ggplot2
223. https://mansthulin.se/posts/ggplot2themes/
224. https://www.sthda.com/english/wiki/ggplot2-themes-and-background-colors-the-3-elements
225. https://bookdown.dongzhuoer.com/hadley/ggplot2-book/theme-elements
226. https://www.jumpingrivers.com/blog/intro-to-theme-ggplot2-r/
227. https://github.com/tidyverse/ggplot2/issues/2938
228. https://stackoverflow.com/questions/68679777/error-when-trying-to-crop-spatial-extent-using-package-sf
229. https://www.volcengine.com/article/12216
230. https://ggplot2-book.org/maps.html
231. https://stackoverflow.com/questions/72575803/ggplot2-spatial-map-add-a-line-segment-on-to-the-border
232. image.jpg
233. https://datascience.blog.wzb.eu/2019/04/30/zooming-in-on-maps-with-sf-and-ggplot2/
234. https://ggplot2.tidyverse.org/reference/ggsf.html
235. https://r-spatial.github.io/sf/articles/sf5.html
236. https://stackoverflow.com/questions/75188773/why-is-ggplot-cropping-the-density-estimation-laid-over-my-map
237. https://datacarpentry.github.io/semester-biology/materials/spatial-data-cropping-R/
238. https://github.com/tidyverse/ggplot2/issues/3938
239. https://stackoverflow.com/questions/70724669/why-do-some-xlims-and-ylims-produce-this-error-in-ggplot-and-sf
240. https://github.com/tidyverse/ggplot2/issues/2985
241. https://stackoverflow.com/questions/78031839/ggplot-map-displaying-inaccurate-coordinate-scale
242. https://github.com/tidyverse/ggplot2/issues/2938
243. https://github.com/tidyverse/ggplot2/issues/3531

244. https://r-spatial.org/r/2018/10/25/ggplot2-sf-2.html
245. https://r-spatial.org/r/2018/10/25/ggplot2-sf.html
246. https://github.com/tidyverse/ggplot2/issues/3962
247. https://search.r-project.org/CRAN/refmans/ggplot2/help/geom_sf.html
248. https://ggplot2.tidyverse.org/reference/ggsf.html
249. https://stackoverflow.com/questions/68343004/ggplot-and-geom-sf-not-showing-graticule-and-tick-marks-on-world-map
250. https://stackoverflow.com/questions/67356840/geom-sf-relabeling-graticules-with-expand-false
251. https://datascience.blog.wzb.eu/2019/04/30/zooming-in-on-maps-with-sf-and-ggplot2/
252. https://github.com/tidyverse/ggplot2/issues/4571
253. https://rdrr.io/cran/shadowtext/src/R/geom-shadowtext.R
254. https://stackoverflow.com/questions/75426660/trying-to-understanding-a-geom-smooth-error
255. https://stackoverflow.com/questions/75221136/error-in-geom-text-geom-text-requires-the-following-missing-aesthetics
256. https://stackoverflow.com/questions/74043151/geom-text-in-ggplot-gem-bar-geom-text-requires-the-following-missing-aesthet
257. https://ggrepel.slowkow.com/articles/examples
258. https://ggplot2.tidyverse.org/reference/geom_text.html
259. https://forum.posit.co/t/ggplot2-correct-use-of-aes/1543
260. https://www.youtube.com/watch?v=kgAWKRfXduc
261. https://github.com/tidyverse/ggplot2/issues/4566
262. https://stackoverflow.com/questions/22311652/ggplot2-geom-path-requires-the-following-missing-aesthetics
263. https://cran.r-project.org/web/packages/shadowtext/shadowtext.pdf
264. https://r-graph-gallery.com/275-add-text-labels-with-ggplot2.html
265. https://rdrr.io/cran/shadowtext/src/R/geom-shadowtext.R
266. https://cran.r-project.org/web/packages/shadowtext/vignettes/shadowtext.html
267. https://www.youtube.com/watch?v=hDfauqNWr2o
268. https://rfortherestofus.com/2024/05/shadowtext-ggplot
269. https://cran.r-project.org/web/packages/shadowtext/news/news.html
270. https://stackoverflow.com/questions/56318012/using-ggrepel-and-shadowtext-on-the-same-geom-text
271. https://github.com/GuangchuangYu/shadowtext/issues/4
272. https://ggplot2.tidyverse.org/reference/ggsf.html
273. https://patchwork.data-imaginist.com/reference/plot_layout.html
274. https://stackoverflow.com/questions/68787739/patchwork-collect-legend-with-different-underlying-values
275. https://patchwork.data-imaginist.com/reference/guide_area.html
276. https://cran.r-project.org/web/packages/patchwork/patchwork.pdf
277. https://patchwork.data-imaginist.com/articles/guides/layout.html
278. https://ggplot2-book.org/scales-colour.html

279. https://r-statistics.co/ggplot2-guides-in-R.html
280. https://stackoverflow.com/questions/79663257/how-to-handle-legend-properly-with-patchwork
281. https://ggplot2.tidyverse.org/reference/guide_coloursteps.html
282. https://stackoverflow.com/questions/74586523/patchwork-does-not-collect-guides
283. https://factsinstitute.com/ranking/largest-countries-by-area-ranked/
284. https://www.datapandas.org/ranking/largest-countries-in-the-world
285. https://ufal.mff.cuni.cz/~cinkova/2024/docs/clips.pdf
286. https://www.astons.com/news/top-10-largest-countries-in-the-world-by-area/
287. image.jpg
288. https://www.britannica.com/topic/list-of-the-total-areas-of-the-worlds-countries-dependencies-and-territories-2130540
289. https://www.aneki.com/top_100_largest.html
290. https://en.wikipedia.org/wiki/List_of_countries_and_dependencies_by_area
291. https://www.worldatlas.com/geography/the-largest-countries-in-the-world.html
292. https://www.infoplease.com/geography/largest-countries-world
293. https://www.theglobaleconomy.com/rankings/land_area/
294. https://r-spatial.github.io/sf/reference/geos_measures.html
295. https://stackoverflow.com/questions/44302234/r-sp-unit-of-area-of-polygon/44303016
296. https://github.com/cran/area
297. https://mitchellgritts.com/articles/simple-correct-fast/
298. https://cabuelow.quarto.pub/calculating-areas-with-vector-and-raster-data-in-r/
299. https://cran.r-project.org/web/packages/geofi/vignettes/geofi_spatial_analysis.html
300. https://cran.r-project.org/web/packages/densityarea/vignettes/sf-operations.html
301. https://github.com/loicdtx/sfr/blob/master/R/geom.R
302. https://cran.r-project.org/web/packages/sf/vignettes/sf3.html
303. https://stackoverflow.com/questions/48437569/how-to-crop-a-polygon-from-an-underlying-grid-with-the-r-sf-package
304. https://patchwork.data-imaginist.com/reference/plot_annotation.html
305. https://stackoverflow.com/questions/46287086/how-to-center-ggplot-plot-title/62355132
306. https://github.com/thomasp85/patchwork/issues/143
307. https://github.com/thomasp85/patchwork/issues/43
308. https://patchwork.data-imaginist.com/articles/guides/annotation.html
309. https://www.r-bloggers.com/2022/03/self-documenting-plots-in-ggplot2/
310. https://github.com/thomasp85/patchwork/issues/97
311. https://blog.csdn.net/gitblog_00150/article/details/148862953
312. https://github.com/thomasp85/patchwork/issues/1
313. https://stackoverflow.com/questions/73562383/annotate-combined-subplots-patchworks-as-single-plots-using-patchwork

314. https://ggplot2.tidyverse.org/reference/theme.html
315. https://ggplot2.tidyverse.org/reference/guide_colourbar.html
316. https://github.com/tidyverse/ggplot2/wiki/legend-attributes
317. https://tidyverse.org/blog/2024/02/ggplot2-3-5-0-legends/
318. https://www.sthda.com/english/wiki/ggplot2-legend-easy-steps-to-change-the-position-and-the-appearance-of-a-graph-legend-in-r-software
319. https://ggplot2-book.org/themes.html
320. https://r-graph-gallery.com/239-custom-layout-legend-ggplot2.html
321. https://ggplot2.tidyverse.org/articles/faq-customising.html
322. https://genchanghsu.github.io/ggGallery/posts/2022-10-01-post-20-ggplot-legend-tips-series-no7-working-on-legends-with-theme/
323. https://stackoverflow.com/questions/45617322